



Université Internationale
de Casablanca

UNIVERSITÉ RECONNUE PAR L'ÉTAT

USD/MAD FOREX Shorting Simulator

Une Plateforme de Prédiction des Prix en C++

Par Adimi Abdessamad , Zouhair Hnida — Module : OOP — Professor : BENCHRIF manal

Présentation de l'Agenda

01

Contexte & Problématique

Exploration du marché USD/MAD et des défis actuels (5 minutes).

03

Démonstration Live

Visualisation du simulateur en action (3 minutes).

02

Architecture Technique

Détails sur la conception et les fonctionnalités clés (7 minutes).

04

Analyse & Potentiel Business

Évaluation du marché et des perspectives d'avenir (5 minutes).

Problématique

**Absence d'intégration ludique entre
programmation C++ et analyse financière
pour débutants**

Notre Solution : Un Simulateur C++ Unique

Un simulateur complet en C++ conçu pour démocratiser l'apprentissage du trading USD/MAD.

30 Jours Réalistes

Simulation fidèle des mouvements USD/MAD sur 30 jours pour une expérience plus précise .

6 Rounds de Prédiction

Progression étape par étape avec 6 rounds de prédiction pour affiner les stratégies.

P&L Algorithmique

Calcul précis des profits et pertes grâce à un algorithme intégré.

Gestion de Portefeuille

Suivi automatique et intelligent du portefeuille pour une gestion simplifiée.

Double Valeur : Débutants & Professionnels

Notre simulateur offre une double proposition de valeur, s'adressant à la fois aux novices et aux experts du trading.

Pour les Débutants

Un environnement d'entraînement **sans risque** pour acquérir les bases du trading

- Prédiction USD/MAD réaliste
- Gestion positions LONG/SHORT
- Calcul P&L automatique
- 6 rounds progression
- Wallet virtuel 10k MAD

Pour les Professionnels

Une plateforme pour collecter insights et intelligence collective du marché

- "Votes" prédictions users
- Consensus prix quotidien
- Sentiment bullish/bearish %
- Distribution erreurs prédiction
- API insights payants

"Ce jeu génère des données valorisées, analysées par les traders pour optimiser leurs décisions."

Utilisateurs Cibles : Qui Bénéficiera de Notre Simulateur ?

Quatre profils d'utilisateurs distincts peuvent tirer parti de notre plateforme.



Étudiants C++

Apprentissage de la finance algorithmique et développement de compétences pratiques en C++.A travers la création d'une base modele open source



Étudiants en Finance

Compréhension pratique des marchés financiers sans exposition au risque réel.



Traders Débutants

Développement d'une intuition de marché et validation de stratégies pour "battre le marché".



Traders Professionnels

Obtention d'insights précieux grâce à la "sagesse de la foule" crowd wisness et validation de théories.

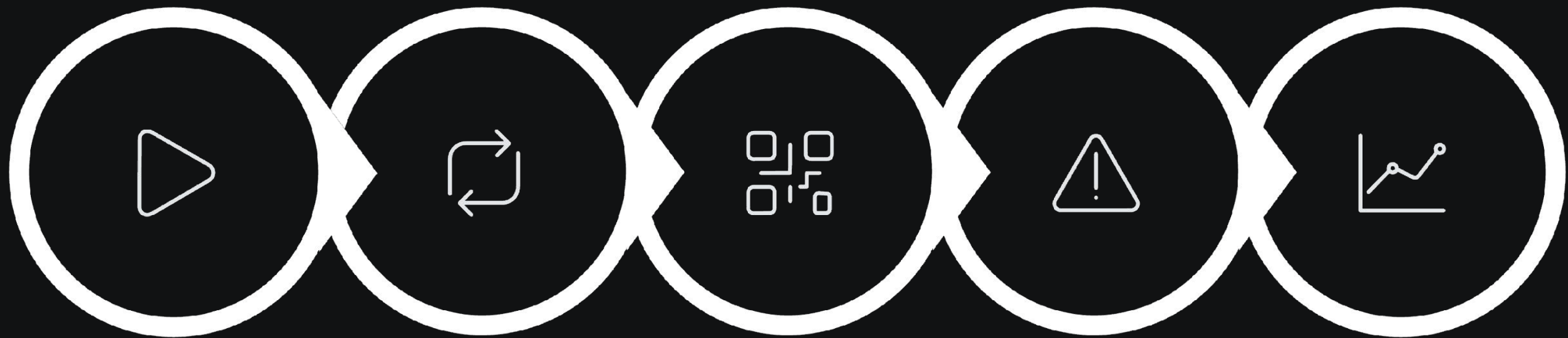
7 Fonctions Modulaires :

Le Cœur du Simulateur

Chaque fonction est conçue pour une tâche spécifique, garantissant modularité et clarté.



Flux Complet du Simulateur USD/MAD



MAIN()

Loop 0-5

playBet()

Validation

showFinalPnL()

Fonction 1 & 2 : MAIN() et PLAYBET()

MAIN() - Le Chef d'Orchestre

Rôle: Lance le jeu et contrôle les 6 rounds

Logique du Code:

```
for(int i=0; i<6; i++)
    playBet()
showFinalPnL()
```

Input: Aucune

Output: Aucune

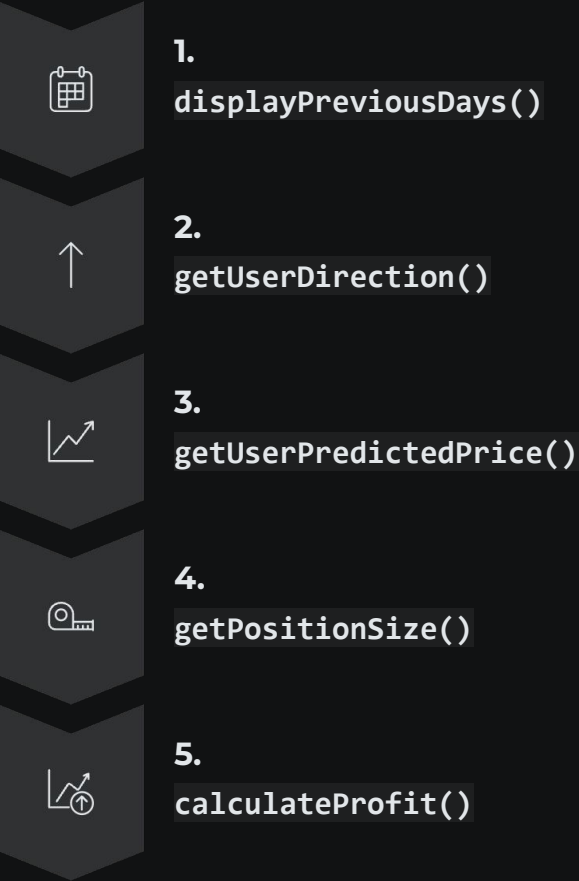
Variables Globales Modifiées: Aucune (contrôle seulement)

Point Clé: Boucle 6 fois, chaque itération = 1 pari

PLAYBET() - Organise 1 Pari Complet

Rôle: Orchestre une séquence complète de prédiction

Séquence (dans cet ordre):



Input: Aucune (utilise betNumber global)

Output: Aucune

Variables Globales Modifiées: betNumber++ (passe au pari suivant)

Point Clé: Appelle 5 fonctions dans un ordre précis

Fonction 3 : DISPLAYPREVIOUSDAYS() - Montre l'Historique

Rôle:

Affiche les 5 derniers jours de prix pour guider la décision.

Paramètres:

Input 1: `daysToShow` = 5 (nombre de jours à afficher)

Input 2: `startDay` = jour actuel - 5 (où commencer)

Flux Logique:

Affiche `=== Previous 5 Days ===` (bordure)

Boucle: `for(i=startDay; i<startDay+5; i++)`

Chaque itération affiche: `cout << "Day " << i << "\t" << dollarPrices[i]`

Affiche bordure finale: `string(50, '=')`

```
// Affiche l'historique des 5 derniers jours
void displayPreviousDays(int daysToShow, int startDay) {
    cout << "=== Previous " << daysToShow << " Days ===" << endl;
    for (int i = startDay; i < startDay + daysToShow; i++) {
        cout << "Day " << i << "\t" << dollarPrices[i] << " MAD"
        << endl;
    }
    cout << string(50, '=') << endl;
}
```

Exemple de Sortie:

```
=== Previous 5 Days ===
```

```
Day 1      10.50 MAD
```

```
Day 2      10.55 MAD
```

```
Day 3      10.48 MAD
```

```
Day 4      10.60 MAD
```

```
Day 5      10.58 MAD
```

```
=====
```

Output:

Aucune (affiche seulement)

Variables Globales Modifiées:

Aucune

Point Clé:

Fonction "read-only" - elle ne modifie rien, affiche seulement

Fonction 4 & 5 : GETUSERDIRECTION() et GETUSERPREDICTEDPRICE()

GETUSERDIRECTION() - Choisit LONG ou SHORT

Rôle: Demande à l'utilisateur de choisir la direction de sa prédiction

Affichage:

```
1. LONG (Achat)
2. SHORT (Vente)
Votre choix:
```

Logique du Code:

```
cin >> choice;
while(choice != 1 && choice != 2) {
    cout << "Invalid! Choose 1 or 2" << endl;
    cin >> choice;
}
return choice;
```

Input: Aucune (saisie utilisateur)

Output: int (1 pour LONG, 2 pour SHORT)

Variables Globales Modifiées: Aucune

Point Clé: Validation en boucle - rejette les entrées invalides

GETUSERPREDICTEDPRICE() - Prix Prévu

Rôle: Collecte la prédiction de prix pour le jour suivant

Affichage:

```
Last price: 10.50 MAD
Predict next price:
```

Logique du Code:

```
cout << "Last price: " << previousPrice << " MAD" << endl;
cout << "Predict next price: ";
cin >> predictedPrice;
return predictedPrice;
```

Input: double previousPrice (dernier prix connu)

Output: double (prix prévu par l'utilisateur)

Variables Globales Modifiées: Aucune

Point Clé: Utilise le prix précédent comme référence

Carte sans titre

[Voir plus](#)

Fonction 6 : GETPOSITIONSIZE() - Montant Parié

Rôle:

Demande et valide le montant que l'utilisateur veut parier

Affichage:

```
Your wallet: 10000 MAD
Position size:
```

Flux Logique:

```
cout << "Your wallet: " << wallet << " MAD"
cout << "Position size: "
cin >> size
if(size > wallet || size <= 0) {
    size = wallet // Force à la limite du portefeuille
}
return size
```

Input: Aucune (saisie utilisateur, utilise wallet global)

Output: double (montant valide à parier)

Variables Globales Modifiées: Aucune

Point Clé: Protège le portefeuille - empêche les paris invalides

Règles de Validation:

- Si size > wallet: réduit à wallet
- Si size <= 0: réduit à wallet
- Sinon: accepte la valeur

Scénarios d'Exemple:

10000 MAD	5000 MAD	5000 MAD	✓
10000 MAD	15000 MAD	10000 MAD (capped)	⚠
10000 MAD	-100 MAD	10000 MAD (invalid)	✗
10000 MAD	0 MAD	10000 MAD (invalid)	✗

Fonction 7 : CALCULATEPROFIT() - LE CŒUR DU JEU

C'est l'algorithme clé qui détermine le profit/perte. Il utilise la prédiction de l'utilisateur versus le prix réel et, surtout, il modifie le portefeuille global. C'est la seule fonction qui change la valeur du portefeuille!

Logique Détaillée - Pas à Pas:

1. Récupère le prix réel: `actualPrice = dollarPrices[dayIndex]`

2. Calcule l'erreur: `erreur = abs(actualPrice - predictedPrice)`

3. Protège contre division par zéro: `if(erreur < 0.001) erreur = 0.001`

4. Calcule multiplicateur: `multiplicateur = posSize / (erreur * 100)`

5. Détermine direction prédite: `predictedUp = (predictedPrice > previousPrice)`

6. Détermine direction réelle: `actualUp = (actualPrice > previousPrice)`

7. Vérifie si direction correcte: `bonneDirection = (predictedUp == actualUp)`

8. Calcule profit: `profit = bonneDirection ? multiplicateur : -multiplicateur`

9. MODIFIE PORTEFEUILLE: `wallet += profit`



10. Affiche tous les détails

INPUTS:

- `double posSize`: montant parié
- `double predictedPrice`: prix prévu par l'utilisateur
- `double previousPrice`: prix du jour précédent
- `int dayIndex`: quel jour (0-29)

Exemple de Calcul:

```
previousPrice = 10.50
predictedPrice = 10.60 (utilisateur prédit HAUSSE)
actualPrice = 10.65 (réalité: HAUSSE)
posSize = 1000 MAD

erreur = |10.65 - 10.60| = 0.05
multiplicateur = 1000 / (0.05 * 100) = 1000 / 5 = 200
predictedUp = (10.60 > 10.50) = TRUE
actualUp = (10.65 > 10.50) = TRUE
bonneDirection = TRUE
profit = 200 MAD

wallet = 10000 + 200 = 10200 MAD ✓
```

Points Clés:

- Plus l'erreur est petite, plus le profit est grand
- Direction correcte = profit positif
- Direction incorrecte = perte (profit négatif)
- C'est la SEULE fonction qui modifie wallet



Fonction 8 : SHOWFINALPNL() - Résultats Finaux

Rôle:

- Affiche les statistiques de performance finales
- Résume le voyage complet du simulateur
- Montre le profit/perte total et le pourcentage de retour

LOGIC:

`totalPnL = wallet - 10000` (profit/perte total)

`returnPercent = (totalPnL / 10000) * 100` (pourcentage de retour)

Affiche tableau final bordé avec tous les détails

INPUT:

Aucune (utilise wallet global)

OUTPUT:

Aucune (affiche seulement)

VARIABLES GLOBALES MODIFIÉES:

Aucune

POINT CLÉ:

Fonction finale - affiche le résultat complet du jeu

Exemple de Sortie:

```
=====
                        FINAL RESULTS
=====

Starting Wallet:      10000.00 MAD
Final Wallet:         10850.50 MAD
Total P&L:            +850.50 MAD
Return %:             +8.51%
=====
```

Guide d'Interprétation:

- If totalPnL > 0: Profit! (positive return)
- If totalPnL < 0: Loss (negative return)
- If totalPnL = 0: Break even

Scénarios d'Exemple:

10850.50 MAD	+850.50 MAD	+8.51%	Profit ✓
9500 MAD	-500 MAD	-5%	Perte (loss)
10000 MAD	0 MAD	0%	Équilibre (break even)

Flux Visuel Complet : De MAIN() à SHOWFINALPNL()

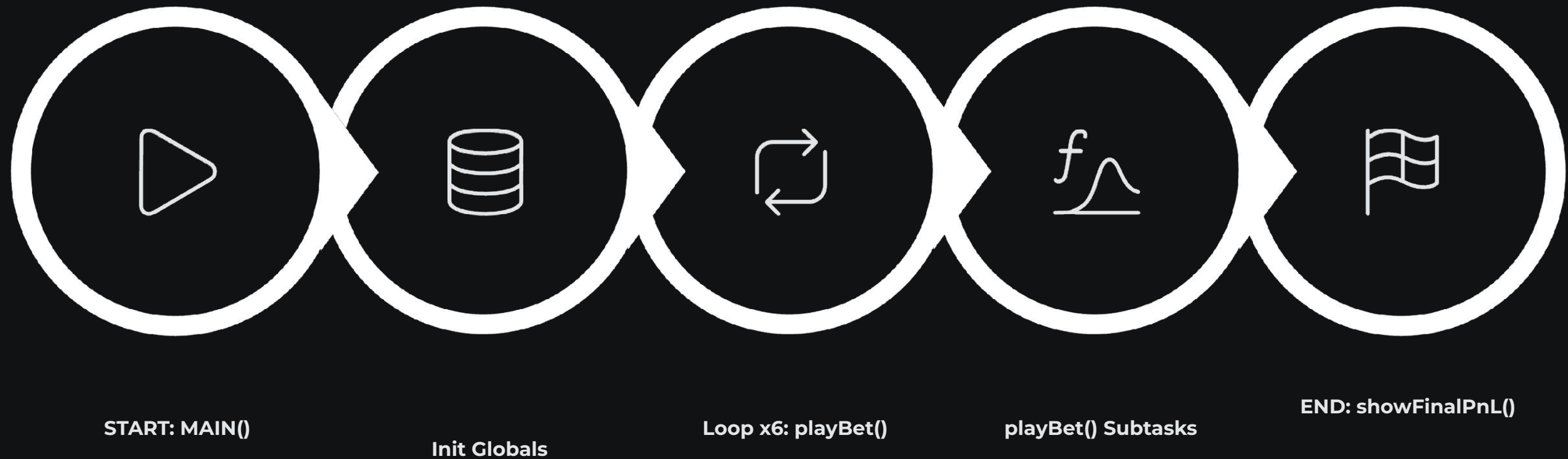


Tableau Récapitulatif : Les 8 Fonctions en Un Coup d'Œil

#	Fonction	Rôle	Input	Output	Clé
1	MAIN()	Chef d'orchestre, boucle 6 fois	Aucune	Aucune	Contrôle seulement
2	PLAYBET()	Orchestre 1 pari complet	Aucune	Aucune	Appelle 5 fonctions
3	displayPreviousDays()	Affiche 5 jours précédents	daysToShow, startDay	Aucune	Read-only
4	getUserDirection()	Choisir LONG ou SHORT	Aucune	int (1 ou 2)	Validation en boucle
5	getUserPredictedPrice()	Collecte prix prévu	previousPrice	double	Référence prix précédent
6	getPositionSize()	Valide montant parié	Aucune	double	Protège portefeuille
7	calculateProfit()	CŒUR: calcule P&L	4 paramètres	Aucune	SEULE qui change wallet
8	showFinalPnL()	Affiche résultats finaux	Aucune	Aucune	Résumé complet

Variables Globales:

wallet: Commence à 10000, change uniquement dans calculateProfit()

betNumber: 0→1→2→3→4→5 (s'incrémente dans playBet())

dollarPrices[30]: Tableau de prix (lecture seule)

Ordre d'Exécution:

MAIN() → Boucle 6 fois → **playBet()** → [displayPreviousDays() → getUserDirection() → getUserPredictedPrice() → getPositionSize() → calculateProfit()] → **showFinalPnL()** → FIN

Catégories de Fonctions:

CONTROL: MAIN(), playBet()

DISPLAY: displayPreviousDays(), showFinalPnL()

INPUT: getUserDirection(), getUserPredictedPrice(), getPositionSize()

CALCULATION: calculateProfit()

Bibliothèques C++ Standard : Efficacité et Simplicité

Le simulateur utilise uniquement des bibliothèques C++ standard pour une portabilité et une maintenance optimales.

<iostream>

Gère les entrées/sorties console (cin, cout).

<iomanip>

Permet le formatage numérique (fixed, setprecision).

<string>

Manipule les chaînes de caractères pour l'affichage.

<cmath>

Utilisé pour des opérations mathématiques comme
abs().

Potentiel Stratégique - De Console à SaaS

Évolution prototype C++ → plateforme monétisable

PHASE 1 - Web + Leaderboard (3 mois)

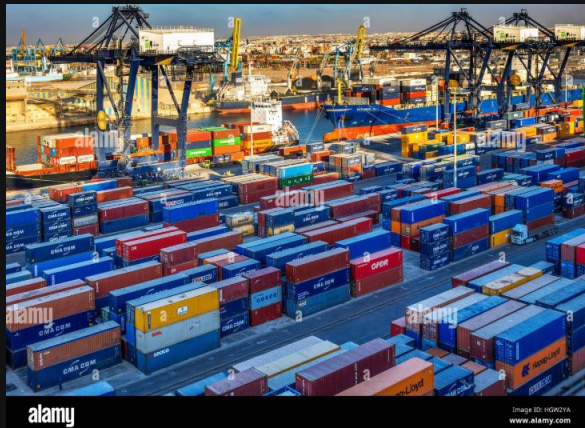
- Frontend React: Users online
- Backend C++ réutilisé
- DB MySQL: Prédictions users
- Leaderboard: Top 10 wallets
- CSV Export données



PHASE 2 - Insights Pros (6 mois)

- API Live: USD/MAD réel
- Consensus temps réel
- Alertes "Crowd Shift"
- API Payante traders
- Multi-devises MAD

Cas d'Usage Concrets - Maroc



MOHAMED - Importateur Chine

"Je dois payer fournisseur 50k\$.
Potentiel coût USD/MAD énorme.
Avec consensus users: 78% voient
10.24 MAD jeudi. → J'attends jeudi
= ÉCONOMIE 2k MAD!"



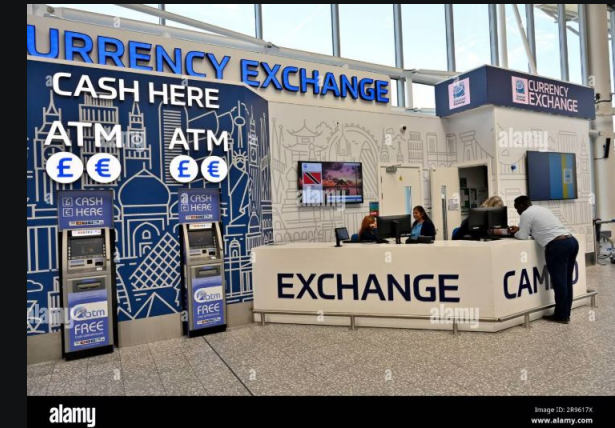
AHMED - Trader Local

"Consensus users: 65% prédisent
baisse. Je trade contrarian =
VENDRE dollar. +15% performance
vs consensus = PROFIT!"



FATIMA - Data Scientist

"Dataset 10k prédictions
users/jour. Je train ML model sur
crowd wisdom. Accuracy +12% vs
solo predictions."



YOUSSEF - Voyageur USA

"Je pars NYC. Consensus voit dollar
10.28. J'achète MAD maintenant →
ÉCHANGE favorable."

Intelligence collective(crowd wisdom) = Décisions optimisées

Conclusion et Prochaines Étapes

Nous vous remercions sincèrement pour votre attention et votre intérêt pour notre simulateur USD/MAD.

Ce projet, démarré comme un prototype C++, démontre un potentiel stratégique considérable. De l'optimisation des décisions financières individuelles à une future plateforme SaaS destinée aux professionnels marocains, nous envisageons une évolution ambitieuse.

Nous sommes convaincus que cette solution peut transformer la gestion des risques de change et offrir des opportunités uniques dans le contexte économique du Maroc.

N'hésitez pas à nous contacter pour toute question ou collaboration.